

OPERATING SYSTEMS (UE25CS242A)

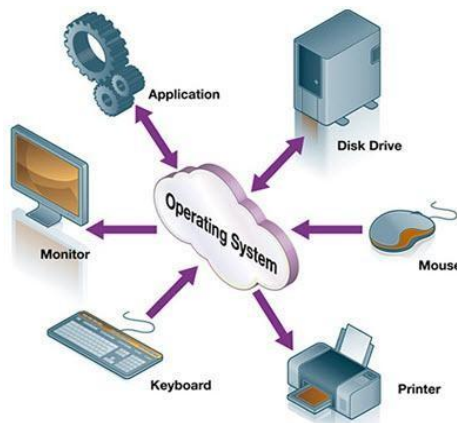
Unit 1 – Introduction & Computer System Organization

Introduction

Operating Systems (OS) form the backbone of modern computing systems. Every application program, from a simple calculator to complex AI software, relies on the operating system to interact with the hardware. Without an operating system, application developers would need to write hardware-specific code, making software development extremely complex and impractical.

Need for an Operating System

If an operating system were not developed, application developers would need to directly interact with hardware components such as the CPU, memory, disk, and I/O devices. This would require detailed knowledge of hardware architecture, device protocols, and low-level instructions.



The operating system abstracts hardware complexity and provides a standardized interface, allowing developers to focus on solving problems rather than managing hardware.

General Definition of an Operating System

An Operating System is a program that acts as an intermediary between the user and the computer hardware.

It provides a user-friendly environment in which users can easily develop and execute programs.

Without an operating system:

- Hardware knowledge would be mandatory for programming
- Software portability would be extremely limited
- Resource conflicts would be difficult to manage

Genesis of Operating Systems

Early computers did not have operating systems. Programs were loaded manually, and only one program could run at a time. Over time, as hardware became more powerful and users demanded multitasking, operating systems evolved to manage resources efficiently and provide better usability.

Operating System Goals

The primary goals of an operating system are:

- Execute user programs and make solving user problems easier
- Make the computer system convenient to use
- Use hardware efficiently
- Manage system resources, including:
 - Memory
 - Processor(s)
 - Input/Output devices

An effective OS balances user convenience with efficient hardware utilization.

Why Study Operating Systems?

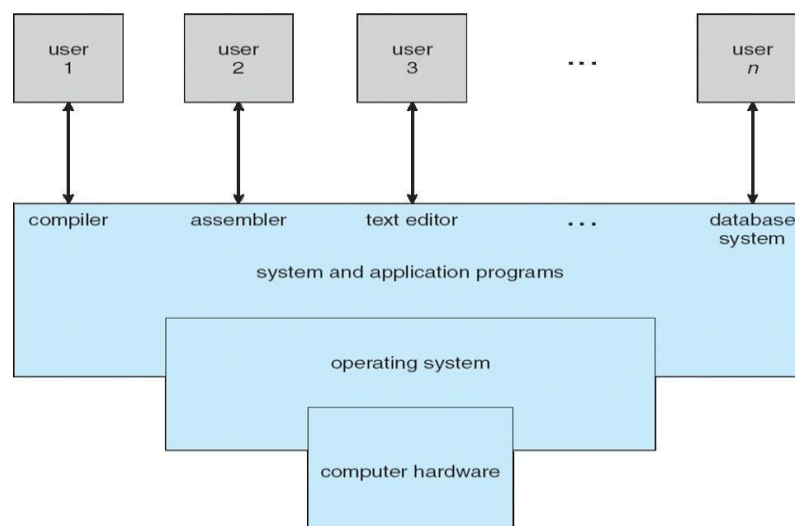
Although only a small percentage of computer professionals design or modify operating systems, almost all software runs on top of an OS.

Understanding operating systems is crucial for:

- Writing efficient programs
- Developing secure applications
- Debugging performance issues
- Understanding resource behavior such as memory usage and CPU scheduling

Knowledge of OS fundamentals leads to proper, efficient, effective, and secure programming.

Four Components of a Computer System (Abstract View)



A computer system can be divided into four major components:

1. **Hardware** - Provides basic computing resources such as:
 - CPU
 - Main memory

- I/O devices
- 2. **Operating System** - Controls and coordinates the use of hardware among various applications and users.
- 3. **Application Programs** - Define how system resources are used to solve user problems
Examples:
 - Word processors
 - Compilers
 - Web browsers
 - Database systems
 - Video games
- 4. **Users** - Can be people, machines, or other computers.

Computer System Structure

The operating system sits between hardware and application programs, ensuring safe and efficient usage of system resources.

What Operating Systems Do (User vs System View)

From the **user's perspective**:

- Desire convenience
- Want ease of use
- Expect good performance
- Do not care much about resource utilization

From the **system's perspective**:

- Especially in shared systems (mainframes, minicomputers)
- Must ensure fair resource allocation
- Must maximize utilization of:
 - CPU time
 - Memory
 - I/O devices
- No single user should monopolize resources

What Operating Systems Do (Different Environments)

- **Dedicated systems (workstations):** Have dedicated resources but still rely on shared servers (file, print, compute servers)
- **Handheld devices:** Resource-constrained, optimized for:
 - Battery life
 - Usability

- **Embedded systems:**
Often have little or no user interface Examples:
 - Automobiles
 - Medical devices
 - Home appliances

Operating systems are designed to balance usability and resource efficiency based on the system type.

System View of Operating System

From a system viewpoint, the operating system serves two major roles:

1. **Resource Allocator**
 - Manages all system resources
 - Resolves conflicting resource requests
 - Ensures fairness and efficiency
2. **Control Program**
 - Controls execution of programs
 - Prevents errors
 - Prevents improper use of the computer

Defining the Operating System

The fundamental goal of a computer system is to execute user programs and solve user problems.

The operating system combines:

- Resource allocation
- Control mechanisms

into a single piece of software.

The kernel is:

- The core part of the OS
- The only program running at all times

Everything else is either:

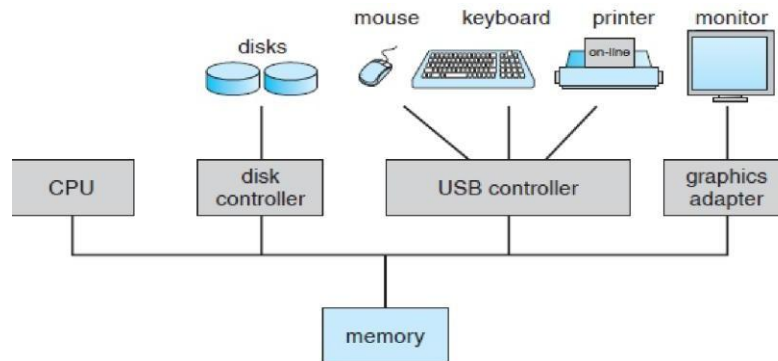
- A system program (ships with OS)
- An application program

Computer System Organization

A computer system consists of:

- One or more CPUs

- Device controllers
- Shared main memory
- Common system bus



The CPU and device controllers can execute **concurrently**, competing for memory cycles. A memory controller synchronizes access to shared memory.

A Modern Computer System

Modern systems support parallelism, concurrency, and high-speed I/O operations.

Computer System Operation

Key points:

- CPU and I/O devices execute concurrently
- Each device controller manages a specific device
- Device controller contains:
 - Local buffer
 - Registers for control actions
- Data transfer occurs:
 - Device → Controller buffer → Main memory

After completing an operation, the device controller signals the CPU by generating an interrupt.

Booting the System

When the system starts:

- The first program executed is the Bootstrap program
- Stored in ROM or EEPROM
- Also called firmware

Functions of bootstrap:

- Initializes CPU registers
- Initializes memory
- Initializes device controllers

- Loads the operating system kernel into memory

After booting:

- The first user-level process created is **init**
- init waits for events and starts system services

Interrupts

An operating system is interrupt-driven.

- Interrupt transfers control to an Interrupt Service Routine (ISR)
- Uses an interrupt vector containing addresses of service routines
- CPU state (program counter, registers) is saved

A trap or exception is a software-generated interrupt caused by:

- Errors
- User requests (system calls)

Interrupt Handling

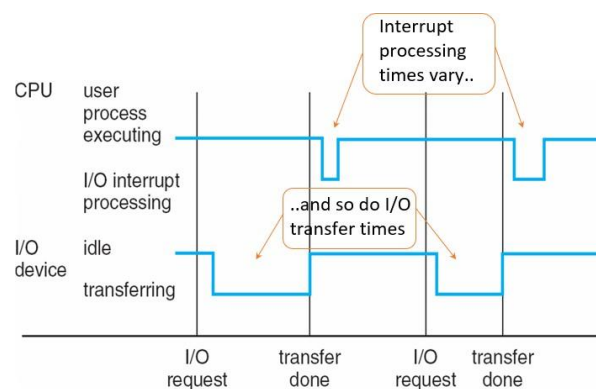
Steps in interrupt handling:

1. Save current CPU state
2. Identify interrupt type
3. Execute appropriate interrupt service routine
4. Restore CPU state
5. Resume execution

Interrupt identification methods:

- Polling
- Vectored interrupt systems

Interrupt Timeline



Shows how a single process performing output is interrupted and resumed.

Storage Structure – Main Memory

Main memory:

- Only large storage directly accessible by CPU
- Random Access Memory (RAM)
- Typically volatile

Implemented using DRAM.

Other memory types:

- ROM
- EEPROM (used in smartphones to store factory-installed programs)

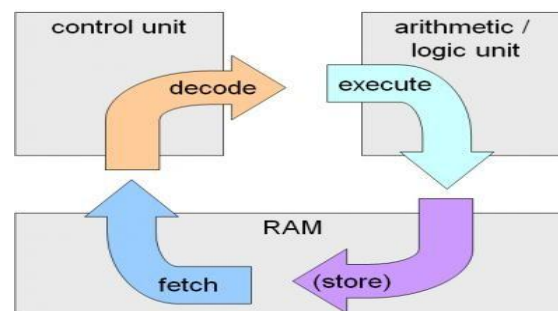
Program Execution Model

Typical instruction execution follows the Fetch–Decode–Execute cycle:

1. Fetch instruction from memory
2. Decode instruction
3. Execute instruction

This repeats until program termination.

This model is known as the Von Neumann architecture.



Secondary Storage

Secondary storage:

- Extension of main memory
- Non-volatile
- Large capacity

Types:

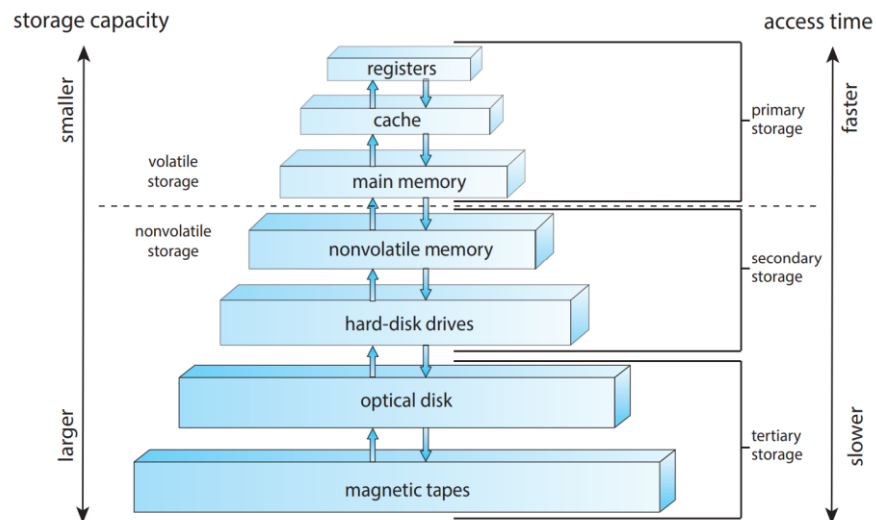
- Hard disks
 - Platters, tracks, sectors
 - Managed by disk controller
- Solid State Drives (SSD)
 - Faster
 - Non-volatile

- Flash memory
 - Used in cameras, PDAs

Storage Hierarchy

Storage systems are organized based on:

- Speed
- Cost
- Volatility



Caching

Caching is a fundamental principle used at many levels:

- Hardware
- Operating system
- Software

Key ideas:

- Frequently used data copied to faster storage
- Cache checked first
- If data not present → fetched from slower storage

Challenges:

- Cache size
- Replacement policy
- Cache management

I/O Structure

- Storage is treated as an I/O device

- Large portion of OS code manages I/O
- Focus on reliability and performance

Components:

- CPUs
- Multiple device controllers
- Common system bus

Each device controller has:

- Device driver
- Uniform interface to kernel

I/O Operation Flow

- Device controller maintains local buffer
- Transfers data between device and memory
- CPU resumes execution after I/O completion interrupt

Two models:

1. CPU waits for I/O completion
2. CPU continues execution while I/O proceeds

Direct Memory Access (DMA)

Used for high-speed I/O devices.

- Data transferred directly between device and main memory
- CPU involvement minimized
- Only one interrupt per block transfer

Improves performance significantly.